# HR Directory (직원 명부) — AI Mentor 요구사항

ODC (OutSystems Developer Cloud) 버전 · ecoletree · 2026-07 · Mentor 업로드용

**Mentor Web (앱 생성)** **Mentor Studio (로직/수정)** **PostgreSQL (Aurora)**

> 이 문서는 원본 **O11 솔루션 시나리오를 ODC에 맞게 전환**한 요구사항서입니다. Mentor에 이 PDF를 업로드한 뒤, 각 프롬프트를 **순서대로 하나씩** 입력하세요. 작성 원칙: 영어 프롬프트 권장 · Goal→Context→Details · 엔티티 우선 · 한 프롬프트=한 관심사 · 수정은 "추가"로 표현 · 패턴 어휘 · PII 금지. 실행 후 결과는 **ODC Studio에서 직접 확인**(성공 메시지 맹신 금지).

## 0. 앱 개요

**앱 이름** HR Directory (직원 명부) · **핵심 화면 2개** EmployeeList(서버액션 테이블·동적정렬·페이지네이션), EmployeeDetail(프로필 사진 업로드/다운로드·저장) · **역할** Manager=조회, Administrator=등록/수정 · **데이터** 직원 약 58명(Excel Bootstrap 시드)

### 도구 분담

| 작업 | 담당 | 비고 |
|---|---|---|
| 엔티티·화면·역할·테마·대시보드 | **Mentor Web** (Portal) | 테마/검증은 생성 시점에만 |
| 서버액션·Aggregate·클라이언트 로직 | **Mentor Studio** (ODC Studio 내) | 1개 단위 원자적 요청이 성공률 ↑ |
| 정렬/페이지네이션 UI 배선, Local Variable, 정렬 아이콘 | **수동** | 필터/정렬/검색 UI는 수동 (Known limitations) |

> ⚠ **O11 → ODC 핵심 차이 (반드시 반영)**
> - **데이터베이스:** ODC는 **AWS Aurora PostgreSQL**. O11(SQL Server)의 `[대괄호]` 식별자·`FETCH NEXT` 문법 대신 **PostgreSQL 문법(LIMIT/OFFSET, "쌍따옴표" 또는 소문자 식별자)** 사용.
> - **역할(Roles):** ODC에서는 **앱 단위(app-specific)**로, 앱 간 공유 불가. Manager/Administrator를 이 앱 안에 정의.
> - **Excel Bootstrap:** ODC Studio에서 엔티티 우클릭 → "Create Action to Bootstrap data from an Excel…" → 액션+구조체+리소스+타이머 자동 생성, **Publish 시점**에 시드.
> - **용어:** Service Studio→**ODC Studio**, 1-Click Publish→**Publish**, 모듈→**앱/라이브러리(asset)**, OutSystems UI→**ODC UI**.

## 1. 데이터 모델 — Employee 엔티티 (11속성)

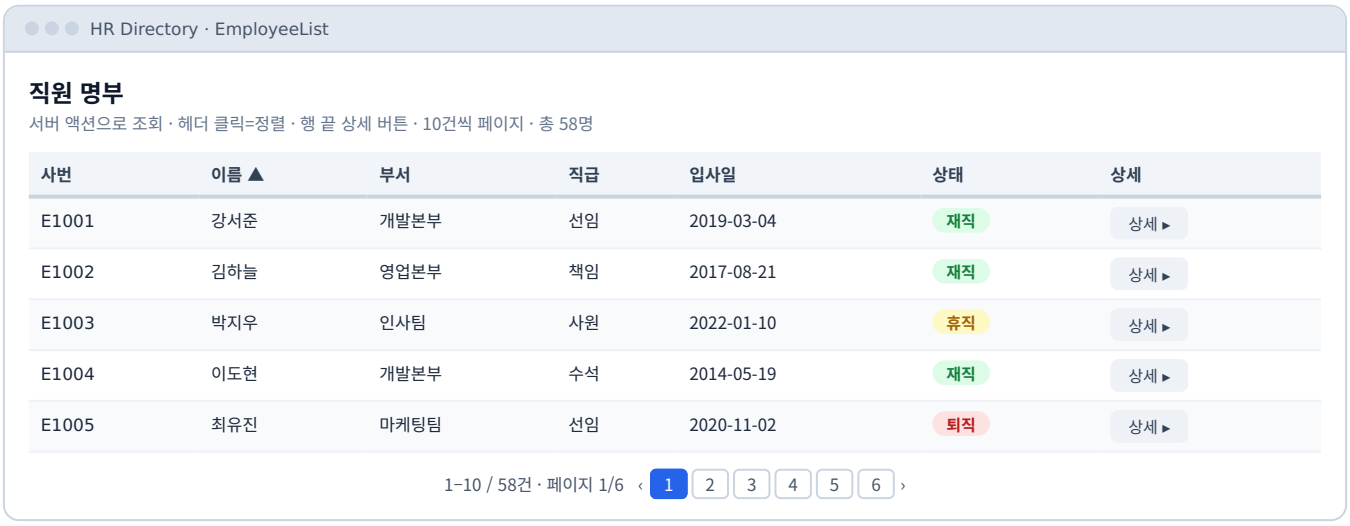
| 속성 | 타입 | 필수 | 설명 |
|---|---|---|---|
| Id | Employee Identifier | 자동 | 자동 생성 키 |
| EmployeeNo | Text | ☒ | 사번 (고유) |
| Name | Text | ☒ | 이름 |
| Department | Text | – | 부서 (개발본부/영업본부/인사팀 등) |
| Position | Text | – | 직급 (사원/선임/책임/수석 등) |
| Email | Email | – | 이메일 (형식 검증) |
| Phone | Phone Number | – | 연락처 |
| HireDate | Date | – | 입사일 (정렬용) |
| Status | Text | – | 재직상태 (재직/휴직/퇴직) |
| ProfilePhoto | Binary Data | – | 프로필 사진 (Upload/Download 대상) |
| PhotoFileName | Text | – | 프로필 사진 파일명 |

⚠ *ProfilePhoto*는 반드시 *Binary Data* 타입. *Text/Image URL*로 두면 *Upload* 위젯과 연결 안 됨.

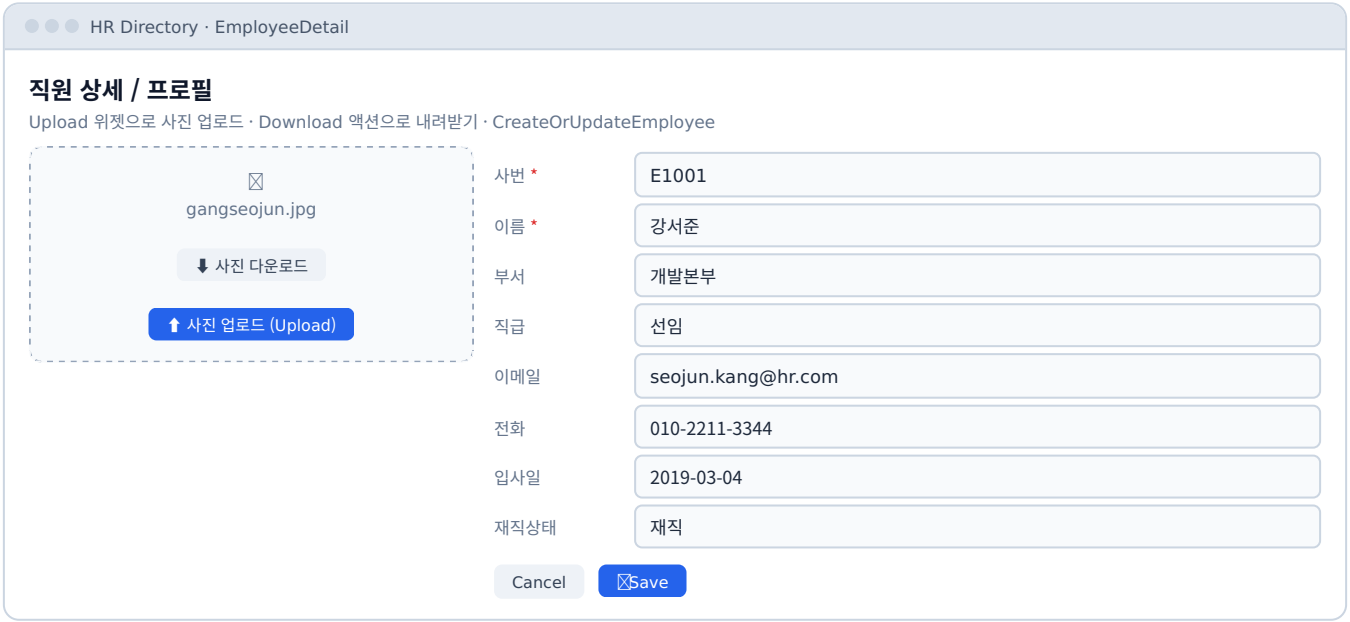
## 2. 완성 화면 (목표 이미지) — Mentor 참고용

아래 두 화면이 최종 완성 형태입니다. Mentor가 정확히 그리도록 이 레이아웃을 목표로 삼으세요.

### 화면 1 · EmployeeList — 서버액션 테이블 · 동적정렬 · 페이지네이션




### 화면 2 · EmployeeDetail — 프로필 사진 업로드/다운로드 · 저장



## 3. PHASE 1 — Mentor Web (앱 생성)

### P1. 앱 + Employee 엔티티 + 역할 + 검증 (최초 프롬프트)

```
Create an HR Directory app to manage Employees. Track an Employee entity with these attributes: EmployeeNo (Text, mandatory), Name (Text, mandatory), Department (Text), Position (Text), Email (Email), Phone (Phone Number), HireDate (Date), Status (Text), ProfilePhoto (Binary Data), PhotoFileName (Text). Define two application roles: Manager (can view the directory) and Administrator (can create and edit employee records). Add email format validation on the Email attribute.
```

> 직원(Employee)을 관리하는 HR Directory 앱을 만들어줘. Employee 엔티티 속성: EmployeeNo(Text,필수), Name(Text,필수), Department(Text), Position(Text), Email(Email), Phone(Phone Number), HireDate(Date), Status(Text), ProfilePhoto(Binary Data), PhotoFileName(Text). 앱 역할 2개 정의: Manager(조회), Administrator(등록/수정). Email 속성에 이메일 형식 검증 추가.

*Goal(앱)→Context(엔티티/속성)→Details(역할·검증).* 역할은 *ODC*에서 앱 단위로 정의됨.

### P2. EmployeeList 화면 (테이블 6컬럼)

```
Create a screen named EmployeeList that lists Employee records in a table with columns: EmployeeNo, Name, Department, Position, HireDate, Status. Sort the table by Name ascending by default.
```

> Employee 레코드를 table로 보여주는 EmployeeList 화면을 만들어줘. 컬럼: EmployeeNo, Name, Department, Position, HireDate, Status. 기본 정렬은 Name 오름차순.

패턴 키워드 *table.* 속성 6개라 *popup(≤5)* 대신 *table* + 상세화면 구조.

### P3. EmployeeDetail 화면 (편집 폼)

```
Create a screen named EmployeeDetail with an input parameter EmployeeId (Employee Identifier, mandatory). Add a form to edit a single Employee with fields: EmployeeNo, Name, Department, Position, Email, Phone, HireDate, Status. Make EmployeeNo and Name mandatory. Add Save and Cancel buttons.
```

> 입력 파라미터 EmployeeId(Employee Identifier, 필수)를 가진 EmployeeDetail 화면을 만들어줘. 단일 Employee 편집 form, 필드: EmployeeNo, Name, Department, Position, Email, Phone, HireDate, Status. EmployeeNo·Name 필수. Save·Cancel 버튼.

### P4. (선택) 요약 대시보드

```
Create a dashboard with a counter for total employees (Count of Employee), a vertical bar chart for headcount by Department (Count grouped by Department), and a pie chart for distribution by Status (Count grouped by Status).
```

> 대시보드를 만들어줘: 총 직원 수 카운터, 부서별 인원 세로 막대 차트, 재직상태별 분포 파이 차트.

대시보드 리스트는 최대 5건·필터 불가 → 전체 탐색은 *EmployeeList*가 담당.

## 4. 데이터 시드 (ODC Excel Bootstrap · 수동)

**PII 금지:** 실명·실제 이메일·전화번호를 프롬프트에 넣지 말 것. 샘플 58명은 Excel Bootstrap으로 시드.

- ODC Studio에서 Employee 엔티티 **우클릭 → "Create Action to Bootstrap data from an Excel..."** → 제공 Excel(약 58행) 선택
- ODC Studio가 부트스트랩 액션 + 구조체 + 리소스 + 타이머를 자동 생성
- **Publish** → 타이머가 Publish 시점에 실행되어 시드됨
- ProfilePhoto는 비어 있는 채 시드 → 상세 화면 Upload로 채움. 50건 이상이어야 페이지네이션(6페이지)·동적정렬 검증 가능

## 5. PHASE 2 — Mentor Studio (로직) · 원자적 요청

### L1. Aggregate — GetEmployeeCount

```
Create an aggregate named GetEmployeeCount with source Employee, used only for the total record count.
```

Source가 Employee인 GetEmployeeCount Aggregate. 전체 건수(.Count) 용도.

### L2. Aggregate — GetEmployeeById

```
Create an aggregate named GetEmployeeById with source Employee, filtered by Employee.Id = EmployeeId, Max Records = 1.
```

Source가 Employee인 GetEmployeeById Aggregate. Filter: Employee.Id = EmployeeId, Max Records = 1.

### L3. Server Action — GetEmployeesPaged (PostgreSQL)

```
Create a server action named GetEmployeesPaged. Inputs: StartIndex (Integer), MaxRecords (Integer), SortColumn (Text),
SortAscending (Boolean). Outputs: Employees (Employee List), TotalCount (Integer). Use a SQL element (PostgreSQL) to
fetch one page:
SELECT {Employee}.* FROM {Employee} ORDER BY @OrderBy LIMIT @MaxRecords OFFSET @StartIndex
The OrderBy SQL parameter must be Expand Inline = Yes, built from a whitelist If() expression on SortColumn (allow only
known columns) plus ASC/DESC from SortAscending, to prevent SQL injection. Set TotalCount from the GetEmployeeCount
aggregate Count (not from the paged SQL count).
```

GetEmployeesPaged 서버 액션. 입력: StartIndex, MaxRecords(Integer), SortColumn(Text), SortAscending(Boolean). 출력: Employees(Employee List), TotalCount(Integer). SQL 요소(PostgreSQL)로 한 페이지 조회: SELECT {Employee}.* FROM {Employee} ORDER BY @OrderBy LIMIT @MaxRecords OFFSET @StartIndex. OrderBy는 Expand Inline=Yes, SortColumn을 whitelist If식으로 매핑(허용 컬럼만)+ASC/DESC (인젝션 방지). TotalCount는 GetEmployeeCount Aggregate의 Count로 채움.

**PostgreSQL 주의:** O11의 `OFFSET @StartIndex ROWS FETCH NEXT @MaxRecords ROWS ONLY` + `[Name]`(대괄호) 대신 → `LIMIT @MaxRecords OFFSET @StartIndex` 사용. whitelist 예: `If(SortColumn="Name","name",If(SortColumn="hiredate","hiredate","name")) + If(SortAscending," ASC"," DESC")` — PostgreSQL 물리 컬럼은 소문자. 대소문자 이슈 시 큰따옴표(`"Name"`)로 감쌈. Aggregate는 offset 미지원이라 SQL 사용.

### L4. Server Action — CreateOrUpdateEmployee

```
Create an entity server action named CreateOrUpdateEmployee that creates or updates an Employee record, including
ProfilePhoto (Binary Data) and PhotoFileName.
```

Employee 레코드를 등록/수정하는 CreateOrUpdateEmployee 엔티티 서버 액션. ProfilePhoto(Binary Data)·PhotoFileName 포함 저장.

## 6. PHASE 3 — 화면 상태·로직

정렬/페이지네이션 *UI* 배선·*Local Variable*·정렬 아이콘은 공식적으로 수동 영역. 로직은 프롬프트로 만들되 위젯 바인딩은 *Studio*에서 직접 확인.

### S1. EmployeeList Local Variables (수동)

StartIndex(Integer=0), PageSize(Integer=10), SortColumn(Text="Name"), SortAscending(Boolean=True), Employees(Employee List), TotalCount(Integer), TotalPages(Integer)

### S2. Screen Action — LoadEmployees

```
On the EmployeeList screen, create a screen action LoadEmployees that calls GetEmployeesPaged with StartIndex, PageSize,
SortColumn, SortAscending, then assigns Employees, TotalCount, and TotalPages = Trunc((TotalCount + PageSize - 1) /
PageSize). Call LoadEmployees on the screen OnInitialize.
```

EmployeeList에 LoadEmployees 스크린 액션. GetEmployeesPaged 호출 후 Assign: Employees, TotalCount, TotalPages=Trunc((TotalCount+PageSize-1)/PageSize). OnInitialize에서 호출.

*OutSystems엔 Ceiling 함수 없음 → 올림은 Trunc((n+d-1)/d).*

## S3. Screen Action — SortBy (동적정렬)

```
On the EmployeeList screen, create a screen action SortBy with input ColumnName (Text). If SortColumn = ColumnName, set SortAscending = not SortAscending; otherwise set SortColumn = ColumnName and SortAscending = True. Then set StartIndex = 0 and run LoadEmployees. Bind each table header On Click to SortBy passing its column name.
```

> EmployeeList에 SortBy 스크린 액션(입력 ColumnName Text). SortColumn=ColumnName이면 방향 토글, 아니면 SortColumn 교체·오름차순. 이후 StartIndex=0, LoadEmployees. 각 헤더 On Click에 SortBy(컬럼명) 연결.

방향 ▲▼ 아이콘은 수동 표시. *SortColumn* 문자열은 *L3 whitelist*와 정확히 일치.

## S4. Client Action — PaginationOnNavigate (ODC UI)

```
On the EmployeeList screen, add an ODC UI Pagination pattern below the table with StartIndex = StartIndex, MaxRecords = PageSize, TotalCount = TotalCount. Create a client action PaginationOnNavigate with input NewStartIndex (Integer) that assigns StartIndex = NewStartIndex and runs LoadEmployees. Bind the pattern OnNavigate event to it.
```

> EmployeeList 테이블 하단에 ODC UI Pagination 패턴: StartIndex=StartIndex, MaxRecords=PageSize, TotalCount=TotalCount. PaginationOnNavigate 클라이언트 액션(입력 NewStartIndex) → Assign StartIndex=NewStartIndex → LoadEmployees. OnNavigate에 연결.

*NewStartIndex* 대입이 빠지면 항상 1페이지만 표시. *StartIndex*는 0-*based*.

## S5. 행 끝 '상세' 버튼 → EmployeeDetail Navigate

```
Add a last column to the EmployeeList table with a "Detail" button that navigates to EmployeeDetail passing EmployeeId = Employees.Current.Id.
```

> EmployeeList 테이블 마지막에 '상세' 버튼 컬럼 → EmployeeDetail로 Navigate, EmployeeId=Employees.Current.Id.

## 7. PHASE 4 — 프로필 사진 & 저장

### D1. Upload 위젯 바인딩 (수동 확인)

- Form 안 Upload 위젯: Filename → ...Employee.PhotoFileName, Binary Content → ...Employee.ProfilePhoto
- 미리보기 Image 위젯: **Type = Binary Data**(External URL 아님) → ProfilePhoto 바인딩
- 저장은 Save의 CreateOrUpdateEmployee에서 함께 커밋

### D2. Screen Action — DownloadPhoto

```
On EmployeeDetail, create a screen action DownloadPhoto that downloads the file with content = GetEmployeeById.List.Current.Employee.ProfilePhoto and filename = GetEmployeeById.List.Current.Employee.PhotoFileName. Only enable the download button when PhotoFileName is not empty.
```

> EmployeeDetail에 DownloadPhoto 스크린 액션. Download 노드: Content=...ProfilePhoto, FileName=...PhotoFileName. 다운로드 버튼은 PhotoFileName이 비어있지 않을 때만 활성화.

### D3. Screen Action — SaveEmployeeOnClick

```
On EmployeeDetail, create a screen action SaveEmployeeOnClick bound to the Save button: if the form is valid, call CreateOrUpdateEmployee with the current employee record, show the message "직원 정보가 저장되었습니다.", then navigate to EmployeeList.
```

> EmployeeDetail Save 버튼에 SaveEmployeeOnClick. Form이 Valid면 CreateOrUpdateEmployee(현재 레코드) → "직원 정보가 저장되었습니다." → EmployeeList로 Navigate.

*HireDate 표시는 FormatDateTime(HireDate,"yyyy-MM-dd") — 대문자 MM(월). 소문자 mm은 분(minutes).*

## 8. 실패 시 재시도 플레이북 & 체크리스트

| 재시도 플레이북 (공식) | 프롬프트 체크리스트 |
|---|---|
| 1. 결과를 ODC Studio에서 직접 확인 | ☐ 영어·Goal→Context→Details·이름 참조·한 관심사·엔티티 우선·패턴 키워드 |
| 2. 영어로 재작성 | ☐ popup≤5 / 대시보드 리스트≤5 제약 충돌 없음 |
| 3. 범위 축소 (엔티티 1개/액션 1개) | ☐ 수정은 "추가"로 / 테마·검증은 생성 시점에 |
| 4. "변경"을 "추가"로 재표현 | ☐ PII 없이 Excel Bootstrap으로 시드 |
| 5. 새 대화 시작 후 문장 재구성 | ☐ PostgreSQL 문법(LIMIT/OFFSET) 사용 |
| 6. Mentor 범위 밖이면 수동 전환 | ☐ 실행 후 Studio에서 직접 확인 |

원본 참고: solution.html (O11 솔루션), mentor_prompt_guide - 복사본.html · ODC 전환: PostgreSQL SQL / Excel Bootstrap 액션 / 앱 단위 역할 / ODC Studio·ODC UI 용어